

System Design Basics

Lesson 1: Introduction to System Design

Introduction to System Design - Study Notes

Key Concepts

1. **Definition & Purpose** – System design is the process of creating a blueprint that defines the architecture, components, and interactions of a software system to meet specified requirements.
2. **Scope & Constraints** – Clarifying what the system must do (functional requirements) and the limits it must operate within (performance, scalability, cost, security, compliance).
3. **High Level Architecture** – Choosing between monolithic, microservices, serverless, or hybrid patterns and mapping out major subsystems, data flows, and external interfaces.
4. **Trade offs & Decision Making** – Evaluating options (e.g., relational vs. NoSQL, caching vs. real time) and documenting the rationale behind each choice.
5. **Scalability & Reliability** – Designing for horizontal scaling, fault tolerance, load balancing, and graceful degradation.

Summary

System design is a structured approach to building robust, scalable, and maintainable software systems. It starts with a clear definition of the problem, the scope of the solution, and the key goals such as performance, availability, and cost efficiency. By breaking the system into logical components, designers can reason about interactions, data flow, and failure modes. The process also involves making informed trade offs—choosing the right storage, messaging, and deployment patterns—while keeping future growth and maintenance in mind.

A classic beginner example is a URL shortener. Even a simple service exposes a REST API to create short links, stores mappings in a database, and redirects users. Yet, designing it for millions of requests reveals challenges in caching, database sharding, and ensuring high availability. This illustrates how a seemingly trivial requirement can evolve into a complex design exercise.

Important Points

- **Clarify Requirements Early** – Functional, non functional, and edge case requirements must be documented before architecture decisions.
- **Define Success Metrics** – Latency, throughput, error rate, and cost are measurable goals that guide design choices.
- **Layered Architecture** – Separate concerns: presentation, business logic, data access, and infrastructure.
- **Data Modeling Matters** – Choose the right data store (SQL, NoSQL, cache) based on access patterns and consistency needs.
- **Scalability Patterns** – Use load balancers, stateless services, and horizontal scaling to handle growth.
- **Fault Tolerance** – Implement retries, circuit breakers, and graceful degradation to maintain

service availability.

- **Monitoring & Observability** – Log, trace, and metric collection are essential for diagnosing issues in production.
- **Iterative Design** – Start simple, then refactor and add complexity as requirements evolve.

Examples

1. URL Shortener

- **Components**: API gateway, short link service, database (e.g., Redis for hot keys, PostgreSQL for persistence), CDN for redirects.
- **Design Decisions**:
 - Use a hash function or sequential ID for generating short codes.
 - Cache popular URLs in Redis to reduce DB hits.
 - Replicate DB across regions to lower latency.
 - Implement rate limiting to prevent abuse.
- **Scalability**: Horizontal scaling of the service behind a load balancer; sharding the database by hash prefix.

2. Simple Chat Application

- **Components**: WebSocket server, message broker (Kafka), user presence service, NoSQL store (Cassandra) for chat history.
- **Key Design Choices**:
 - Persist messages in a write optimized store for quick delivery.
 - Use a publish/subscribe model for real time updates.
 - Store user presence in an in memory store (Redis) for low latency.
- **Reliability**: Ensure at least once delivery with message acknowledgments; use replication for broker and database.

Quick Review

1. What is the primary goal of system design and how does it differ from coding?
2. List three non functional requirements that often drive architectural decisions.
3. Why is caching important in a high traffic service like a URL shortener?
4. Explain the trade off between using a relational database versus a NoSQL database for storing URL mappings.
5. Describe a strategy to achieve high availability for a stateless service.

Further Reading

- **Scalable Architecture Patterns** – Monolith, Microservices, Serverless, Event Driven
- **CAP Theorem & Consistency Models** – Understanding trade offs in distributed data stores
- **Load Balancing & Traffic Shaping** – Round robin, least connections, sticky sessions
- **Observability Fundamentals** – Logging, metrics, tracing, and alerting
- **Designing for Failure** – Circuit breakers, retries, bulkheads

- **Security in System Design** – Authentication, authorization, data encryption, OWASP guidelines

These topics will deepen your understanding and prepare you for more advanced system design challenges.

